Legolcsóbb utak és minimális feszítőfák

Bellman–Ford, Kruskal és Prim algoritmusa

Tartalom

1. Legolcsóbb utak: a feladat
2. A fokozatos közelítés elve
3. Bellman–Ford-algoritmus
4. Minimális feszítőfa: a feladat
5. A feszítőfa növelésének elve
6. Kruskal algoritmusa
7. Prim algoritmusa

Honnan indulunk?

- A **szélességi keresés** megadta a **súlyozatlan** gráfokban a legrövidebb utakat – ott az út hossza az azt alkotó **élek száma** volt.
- Most **súlyozott** gráfokkal dolgozunk: az egyes élekhez súlyokat rendelünk.
- Azt az utat keressük, ahol az azt alkotó élek **súlyainak összege** minimális.

Feltételek

A gráf legyen **súlyozott**, és a vizsgált csúcsok legyenek elérhetők a kezdőcsúcsból.

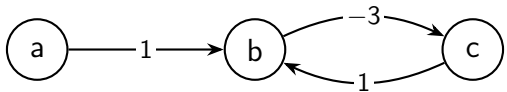
Jelölések

- $c(u, v)$: az $u \to v$ él **súlya** (költsége).
- Egy út **hossza** (költsége): a rajta lévő élek súlyainak összege.
- $\delta(s, v)$: az s-ből v-be vezető **legolcsóbb** út költsége.
  Ha v nem érhető el s-ből, akkor $\delta(s, v) = \infty$.

Megfontolandó

- **Negatív súlyú** éleket is tartalmazó gráfok esete.
- **Negatív összsúlyú köröket** is tartalmazó gráfok esete.

Miért baj a negatív összsúlyú kör?



- A $b \to c \to b$ kör összsúlya $-3 + 1 = -2$.
- Minden körülfordulással **2-vel olcsóbb** utat kapunk.
- Nincs legolcsóbb út: a költség **minden határon alul** csökkenthető.

Negatív él $\neq$ negatív kör

Negatív súlyú él

Önmagában **nem** okoz bajt irányított gráfban.
A legolcsóbb utak léteznek, csak a megoldó algoritmust kell jól megválasztani.

Negatív összsúlyú kör

Ha a kezdőcsúcsból elérhető, akkor a feladatnak **nincs megoldása**.
Egy jó algoritmus ezt **felismeri** és jelzi.

Irányítatlan gráfban egyetlen negatív él már negatív kört is jelent (oda-vissza végigmenve rajta).

A feladat változatai

- **Egy csúcsból az összes többibe** vezető legolcsóbb utak.
  Ezzel foglalkozunk ma.
- **Egy csúcsba** vezető legolcsóbb utak:
  az élek irányításának megfordításával visszavezethető az előzőre.
- **Két adott csúcs** közötti legolcsóbb út:
  az előző megoldás a végén erre is választ ad.
- **Minden csúcspár** közötti legolcsóbb utak:
  léteznek erre specializált megoldások (pl. a Floyd–Warshall-algoritmus).

Gyakorlati alkalmazások

- **Útvonaltervezés:** legrövidebb, leggyorsabb vagy legolcsóbb útvonal.
- **Hálózati forgalomirányítás:** a csomagok továbbítási útvonala.
- **Projekttervezés:** a legkorábbi befejezési idők.
- **Pénzváltási arbitrázs:** a negatív kör itt épp a *nyereséges* váltási sorozatot jelenti – ezért hasznos a felismerése.

Mit tárolunk?

A fokozatos közelítés technikája során két tömböt vezetünk:

$d[v]$ – távolságbecslés

A kiindulópontból eddig talált legolcsóbb út költsége v-be.
Ez mindig egy **felső becslés** a végeredményre: $d[v] \geq \delta(s, v)$.

$\pi[v]$ – megelőző csúcs

Minden csúcsnál eltároljuk, hogy ezen az úton **melyik csúcsból** értük el az adott csúcsot.

A kiinduló állapot

```
eljárás Inicializál(G, s)
  ciklus v ∈ G.Csúcsok
    d[v] ← ∞
    π[v] ← ∅
  ciklus vége
  d[s] ← 0
```

- A kezdőcsúcs esetében $d[s] = 0$ és $\pi[s] = \varnothing$.
- A többi csúcshoz még nem találtunk utat, ezért ott $d = \infty$.

A közelítés (élvizsgálat)

Mikor javíthatunk?

Ha találunk olyan (u, v) élt, amelyre

$$d[u] + c(u, v) < d[v],$$

akkor a v csúcshoz találtunk egy eddiginél **olcsóbb** utat.

```
eljárás Közelít(u, v)
  ha d[u] + c(u,v) < d[v] akkor
    d[v] ← d[u] + c(u,v)
    π[v] ← u
  elágazás vége
```

A közelítés tulajdonságai

- A d értékek **csak csökkennek**, soha nem nőnek.
- A $d[v]$ érték soha nem megy a valódi $\delta(s, v)$ alá – minden $d[v]$ egy **valóban létező** út költsége.
- Ha $d[u] = \infty$, az élt nem kell vizsgálni: u-t még nem értük el.

A kulcskérdés

Melyik **sorrendben** végezzük a közelítéseket?
Megfelelő sorrendben végrehajtva bizonyítható, hogy megtaláljuk a legolcsóbb utat minden csúcshoz.

Az eredmény értelmezése

- $d[v]$ mutatja minden v csúcs esetén az oda vezető legolcsóbb út költségét
  (vagy ∞-t, ha nincs ilyen).
- $\pi[v]$ mutatja a fenti úton a megelőző csúcsot
  (kivéve a kiinduló és az el nem ért csúcsokat).

A tényleges utak előállítása

A célcsúcsból elindulva, a π-ben található adatok alapján **visszafelé** lépkedve előállíthatjuk az egyes csúcsokba vezető utakat. Addig megyünk, amíg a kezdőcsúcshoz nem érünk (ahol π értéke $\varnothing$).

A legolcsóbb utak fája

- Az algoritmus a kezdőcsúcsból **minden elérhető** csúcsba előállítja a legolcsóbb utat.
- A π értékek alapján előállítható egy **fa**, amely ezeket az utakat tartalmazza.
- Hasonló a szélességi keresésnél megismert szélességi fához – csak ott az élek száma, itt az élek összsúlya a minimalizált mennyiség.

Egy láncolt lista elejébe szúrva a csúcsokat az út egyszerűen előállítható.

A Bellman–Ford-algoritmus

Mit tud?

Megoldja az egy csúcsból induló legolcsóbb utak feladatát **negatív súlyú élek** esetén is.

- Ha a kezdőcsúcsból elérhető **negatív összsúlyú kör**, azt **felismeri**,
  és jelzi, hogy a feladatnak nincs megoldása.
- Egyszerű felépítésű: nem kell hozzá prioritásos sor.
- Cserébe lassabb, mint a nemnegatív súlyokra épülő mohó megoldás.

Richard Bellman (1958) és Lester Ford (1956) nyomán.

Az alapötlet

- Nem próbáljuk kitalálni a közelítések **helyes sorrendjét**.
- Helyette: **az összes élt** végigvizsgáljuk, **többször egymás után**.
- Minden ilyen körben (menetben) minden élre elvégezzük a közelítést.

Hány menet kell?

Egy körmentes legolcsóbb út legfeljebb $|V| - 1$ élből áll.
Ezért $|V| - 1$ menet **elegendő**.

Miért elég $|V| - 1$ menet?

- **Állítás:** az i-edik menet után minden olyan csúcs d értéke helyes,
  amelyhez vezet legfeljebb i élből álló legolcsóbb út.
- **Miért?** Legyen a legolcsóbb út $s \to \cdots \to u \to v$,
  és álljon i élből. Ekkor az $s \to \cdots \to u$ szakasz $i - 1$ élből áll.
- Az u csúcs d értéke az indukciós feltevés szerint már az $(i - 1)$-edik
  menet után helyes.
- Az i-edik menetben az (u, v) élre is sor kerül, és a közelítés beállítja v helyes értékét.

Negatív kör nélkül minden legolcsóbb út körmentes, így legfeljebb $|V| - 1$ élű.

A pszeudokód

```
függvény BellmanFord(G, s)
  Inicializál(G, s)
  ciklus i := 1-től |V|-1-ig
    ciklus (u,v) ∈ G.Élek
      Közelít(u, v)
    ciklus vége
  ciklus vége
  ciklus (u,v) ∈ G.Élek        (negatívkör-vizsgálat)
    ha d[u] + c(u,v) < d[v] akkor
      vissza HAMIS
  ciklus vége
  vissza IGAZ, (π, d)
függvény vége
```

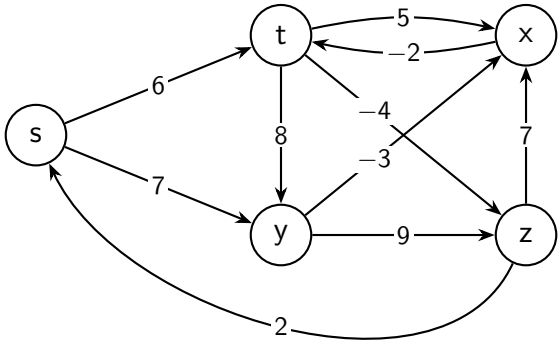
A pszeudokód olvasata

- A **külső ciklus** $|V| - 1$-szer fut le: ennyi menetet végzünk.
- A **belső ciklus** minden menetben végigmegy az **összes élen**.
- A **harmadik ciklus** egy ellenőrző menet: ha itt még mindig javítani lehetne, az negatív összsúlyú körre utal.

Visszatérési érték

HAMIS, ha negatív összsúlyú kör érhető el a kezdőcsúcsból; különben **IGAZ**, és a d, π tömbök tartalmazzák az eredményt.

A példagráf



Kezdőcsúcs: **s**. Három negatív súlyú él is szerepel.

Az élek vizsgálati sorrendje

1.	2.	3.	4.	5.
(t, x) 5	(t, y) 8	(t, z) -4	(x, t) -2	(y, x) -3
6.	7.	8.	9.	10.
(y, z) 9	(z, x) 7	(z, s) 2	(s, t) 6	(s, y) 7

- Minden menetben ebben a sorrendben vizsgáljuk az éleket.
- A sorrend **tetszőleges** lehet – a végeredményt nem befolyásolja, csak azt, hány menet után állnak be az értékek.

Nyomkövetés: a d értékek menetenként

menet	s	t	x	y	z
kezdetben	0	∞	∞	∞	∞

Nyomkövetés: a d értékek menetenként

menet	s	t	x	y	z
kezdetben	0	∞	∞	∞	∞
1.	0	6	∞	7	∞

Nyomkövetés: a d értékek menetenként

menet	s	t	x	y	z
kezdetben	0	∞	∞	∞	∞
1.	0	6	∞	7	∞
2.	0	6	4	7	2

Nyomkövetés: a d értékek menetenként

menet	s	t	x	y	z
kezdetben	0	∞	∞	∞	∞
1.	0	6	∞	7	∞
2.	0	6	4	7	2
3.	0	**2**	4	7	2

Nyomkövetés: a d értékek menetenként

menet	s	t	x	y	z
kezdetben	0	∞	∞	∞	∞
1.	0	6	∞	7	∞
2.	0	6	4	7	2
3.	0	**2**	4	7	2
4.	0	2	4	7	-2

Nyomkövetés: a d értékek menetenként

menet	s	t	x	y	z
kezdetben	0	∞	∞	∞	∞
1.	0	6	∞	7	∞
2.	0	6	4	7	2
3.	0	**2**	4	7	2
4.	0	2	4	7	-2

Négy menet ($= |V| - 1$) kellett: az utolsóban is változott érték.

Az első menet részletesen

él	feltétel	hatás
(t, x)	$d[t] = \infty$	nincs
(t, y)	$d[t] = \infty$	nincs
(t, z)	$d[t] = \infty$	nincs
(x, t)	$d[x] = \infty$	nincs
(y, x)	$d[y] = \infty$	nincs
(y, z)	$d[y] = \infty$	nincs
(z, x)	$d[z] = \infty$	nincs
(z, s)	$d[z] = \infty$	nincs
(s, t)	$0 + 6 < \infty$	$d[t] = 6, \quad \pi[t] = s$
(s, y)	$0 + 7 < \infty$	$d[y] = 7, \quad \pi[y] = s$

Ebben a sorrendben az első menet csak két értéket állít be.

A végeredmény

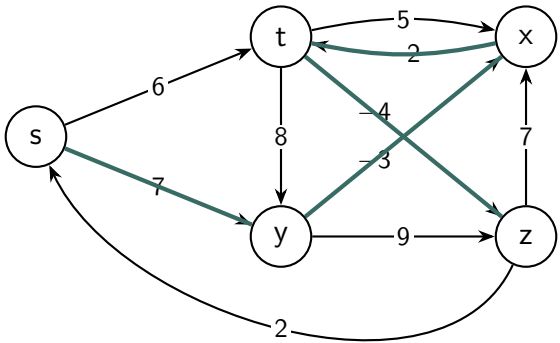
csúcs	d	π
s	0	–
t	2	x
x	4	y
y	7	s
z	-2	t

A π értékekből visszafelé lépkedve:

- y: $s \to y$ (7)
- x: $s \to y \to x$ (4)
- t: $s \to y \to x \to t$ (2)
- z: $s \to y \to x \to t \to z$ (-2)

Ellenőrzés: $7 - 3 = 4$, $4 - 2 = 2$, $2 - 4 = -2$. ✓

A legolcsóbb utak fája



A π által kijelölt élek alkotják a legolcsóbb utak fáját.
A közvetlen $s \to t$ él (súly 6) **nem** része a fának!

A negatívkör-vizsgálat

- A $|V| - 1$ menet után **még egyszer** végigmegyünk az összes élen.
- Ha bármelyik élre még mindig teljesül, hogy

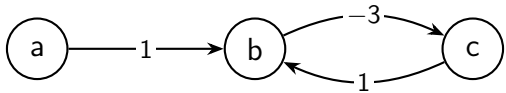
$$d[u] + c(u, v) < d[v],$$

akkor **negatív összsúlyú kör** érhető el a kezdőcsúcsból.

Miért helyes ez?

Negatív kör nélkül minden legolcsóbb út körmentes, tehát legfeljebb $|V| - 1$ élű,
így $|V| - 1$ menet után már semmit nem lehetne javítani.

Példa: negatív összsúlyú kör



menet	a	b	c
kezdetben	0	∞	∞
1.	0	1	-2
2.	0	-1	-4

Az ellenőrző menetben (b, c)-re: $-1 + (-3) = -4 < -4$? Nem.
De (c, b)-re: $-4 + 1 = -3 < -1$ **igaz** $\Rightarrow$ negatív kör.

Futási idő

- Inicializálás: $O(|V|)$.
- $|V| - 1$ menet, menetenként $|E|$ él vizsgálata: $O(|V| \cdot |E|)$.
- Ellenőrző menet: $O(|E|)$.

Összesen

$$O(|V| \cdot |E|)$$

Élsorozatos (éllistás) tárolás esetén a belső ciklus közvetlenül megvalósítható; szomszédsági listáknál minden csúcs minden kivezető élén végigmegyünk.

Gyorsítás: korai leállás

Megfigyelés

Ha egy teljes menetben **egyetlen** d érték sem változott,
akkor a további menetek sem változtatnának semmin.

- Ilyenkor az algoritmus **leállítható**.
- A legrosszabb eset nem javul, de a gyakorlatban sokat segít.
- A példánkban a 4. menet után egy 5. menet már semmit nem változtatna –
  de a negatívkör-vizsgálatot így is el kell végezni.

Mikor melyik algoritmus?

Helyzet	**Mit használjunk?**
nincs negatív él	mohó, prioritásos soros megoldás
van negatív él	**Bellman–Ford**
lehet negatív kör	**Bellman–Ford** (felismeri)
minden csúcspár kell	Floyd–Warshall
súlyozatlan gráf	szélességi bejárás

A nemnegatív súlyokra épülő mohó megoldást (Dijkstra algoritmusát) a Gráfok 1. előadás tárgyalta.

A feladat

Minimális feszítőfa

Olyan **feszítőfát** keresünk, ahol az élek **összsúlya minimális**.

- A gráf legyen **súlyozott**.
- A gráf legyen **irányítatlan**.
- A gráf legyen **összefüggő**.

Emlékeztető: a **feszítőfa** a gráf összes csúcsát tartalmazó, körmentes, összefüggő részgráf. $|V|$ csúcs esetén pontosan $|V| - 1$ éle van.

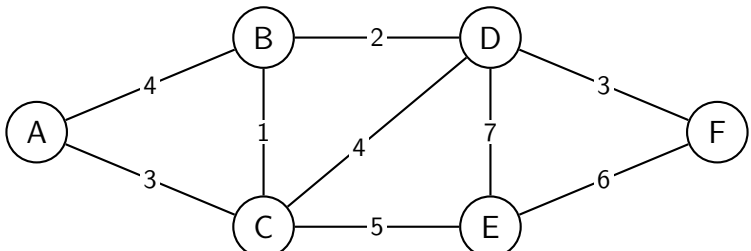
Alapvető észrevételek

- Egy gráfnak **több** minimális feszítőfája is lehet – ezek közül elég az **egyiket** megtalálnunk.
- Ez akkor fordul elő, ha vannak azonos súlyú élek.
- Ha minden élsúly **különböző**, a minimális feszítőfa egyértelmű.
- Az összsúly viszont mindig ugyanaz, bármelyiket is találjuk meg.

Gyakorlati alkalmazások

Áramkörök tervezése, ellátórendszerek (víz, gáz, elektromos hálózat) tervezése,
klaszterezés, képszegmentálás.

A példagráf



Irányítatlan, súlyozott, összefüggő gráf: $|V| = 6$, $|E| = 9$.
A feszítőfának $|V| - 1 = 5$ éle lesz.

A közös váz

- Egy alapállapotból kiindulva **egyesével** adjuk hozzá az éleket a leendő feszítőfához.
- Az algoritmus minden állapotában a feszítőfa egy **részét** tartja nyilván –
  nevezzük ezt az élhalmazt A-nak.

Az invariáns

Az A élhalmaz **mindig része** valamelyik minimális feszítőfának.

Kezdetben $A = \varnothing$, ami triviálisan teljesíti ezt.

Az általános algoritmus

```
függvény ÁltalánosMFF(G)
   A ← ∅
   ciklus amíg A nem feszítőfa
      keressünk egy biztonságos (u,v) élt
      A ← A ∪ {(u,v)}
   ciklus vége
   vissza A
függvény vége
```

- A ciklus pontosan $|V| - 1$-szer fut le.
- A kérdés: mit jelent, hogy egy él **biztonságos**?

Biztonságos él

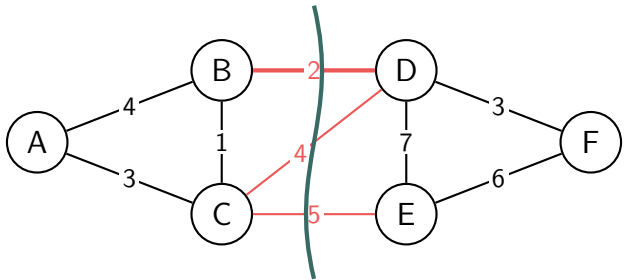
Definíció

Egy él **biztonságos** az A halmazra nézve, ha hozzávéve A-hoz továbbra is fennáll az invariáns:
$A \cup \{(u, v)\}$ is része marad valamelyik minimális feszítőfának.

- Ha minden lépésben biztonságos élt veszünk hozzá, a végén biztosan **minimális** feszítőfát kapunk.
- A két algoritmus abban különbözik, **hogyan keresik** ezt az élt.

Vágás, keresztezés

- **Vágás:** a gráf csúcsainak kettéosztása két csoportba.
- Egy él **keresztezi a vágást**, ha a két végpontja a két **különböző** csoportba került.



A zöld vonal a vágás; a piros élek keresztezik.
A vastagabb piros a **könnyű él**: a keresztezők közül a legkisebb súlyú.

Könnyű él, kikerülő vágás

- Egy él **könnyű egy vágásban**, ha a vágást keresztező élek közül
  neki van a **legkisebb** a súlya.
- Az előző ábrán ez a (B, D) él, 2-es súllyal (a vastagabb piros).
- Egy vágás **kikerüli** az A halmazt, ha az A egyik éle sem keresztezi.

A vágási tétel

Ha A egy minimális feszítőfa része, és egy vágás kikerüli A-t, akkor e vágás **könnyű éle biztonságos** A-ra nézve.

Miért igaz a vágási tétel?

- Legyen T egy minimális feszítőfa, amely tartalmazza A-t, és legyen e a vágás könnyű éle.
- Ha $e \in T$, készen vagyunk.
- Ha $e \notin T$: tegyük hozzá e-t T-hez. Így keletkezik egy **kör**, amely a vágást legalább még egyszer keresztezi – legyen ez az él f.
- Mivel e könnyű él: $c(e) \leq c(f)$.
- Cseréljük ki: $T' = T - \{f\} + \{e\}$ is feszítőfa, és $c(T') \leq c(T)$.
- Tehát T' is minimális, és tartalmazza $A \cup \{e\}$-t. ■

Kruskal algoritmusa – az ötlet

- Vegyük sorra az éleket **növekvő súly** szerint.
- Egy élt akkor veszünk be, ha a két végpontja **eddig különböző** komponensbe tartozott – különben kört zárna.
- Az A halmaz így egy **erdő**, amely fokozatosan összeáll egy fává.

Miért biztonságos a bevett él?

Tekintsük azt a vágást, amely az él egyik végpontjának komponensét választja el a többitől. Ez kikerüli A-t, és a vizsgált él a legkisebb súlyú, amely keresztezi – tehát könnyű él.

Kruskal – a pszeudokód

```
függvény Kruskal(G)
  A ← ∅
  ciklus x ∈ G.Csúcsok
    Halmaz-létrehozás(x)
  ciklus vége
  rendezzük G.Élek-et súly szerint növekvően
  ciklus (u,v) ∈ G.Élek (súly szerint)
    ha Tartalmazó-halmaz(u) ≠ Tartalmazó-halmaz(v) akkor
      A ← A ∪ {(u,v)}
      Halmaz-összevonás(u, v)
    elágazás vége
  ciklus vége
  vissza A
függvény vége
```

A segédműveletek

Halmaz-létrehozás(x)

Létrehoz egy új halmazt, és elhelyezi benne az x elemet (ami egy csúcs).

Tartalmazó-halmaz(x)

A korábban létrehozott halmazok közül megkeresi azt, amelyikben x szerepel.
Csak egy ilyen halmaz lehet, a visszatérési érték ez a halmaz.

Halmaz-összevonás(A, B)

A korábban létrehozott halmazok közül összevon kettőt.

A segédműveletek megvalósítása

- **Tényleges halmaz adatszerkezetekkel:**
  halmazobjektumokkal,
  láncolt listával – az összevonásnál ez praktikus.
- A halmazok maximális száma és maximális elemszáma ismert (mindkettő $|V|$), így **tömbök** is egyszerűen használhatók.
- **Egyszerűbb megoldás:** halmazok helyett minden csúcshoz rendeljünk
  egy **azonosítót**, hogy melyik halmazban van. Az összevonás ekkor
  csak ennek az azonosítónak a másolása.

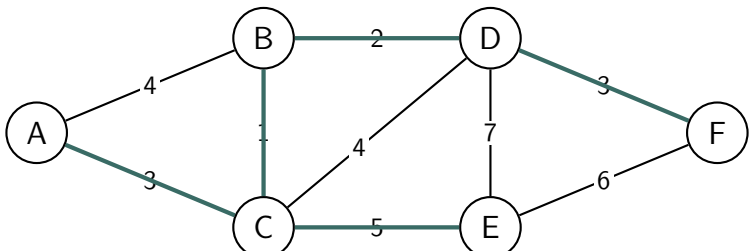
A hatékony megvalósítás neve: *unió-holvan* (union-find) adatszerkezet.

Kruskal – nyomkövetés

él	súly	döntés	összsúly
(B, C)	1	bevesszük	1
(B, D)	2	bevesszük	3
(A, C)	3	bevesszük	6
(D, F)	3	bevesszük	9
(A, B)	4	kört zárna	–
(C, D)	4	kört zárna	–
(C, E)	5	bevesszük	**14**

Öt él bekerült $\Rightarrow$ kész. A maradék (E, F) 6 és (D, E) 7 élt már nem is kell vizsgálni.

Kruskal – az eredmény



A minimális feszítőfa összsúlya: $1 + 2 + 3 + 3 + 5 = \mathbf{14}$.

Kruskal – futási idő

- Az élek rendezése: $O(|E| \log |E|)$.
- $|V|$ darab halmaz-létrehozás.
- $|E|$ darab halmazkeresés és legfeljebb $|V| - 1$ összevonás.

Összesen

$$O(|E| \log |E|) = O(|E| \log |V|)$$

A rendezés dominál. (Mivel $|E| \leq |V|^2$, ezért $\log |E| \leq 2 \log |V|$.)
Hatékony unió-holvan adatszerkezettel a halmazműveletek összköltsége elhanyagolható.

Prim algoritmusa – az ötlet

- Egyetlen csúcsból indulunk, és a fát **egy darabban** növeljük.
- Minden lépésben azt a csúcsot csatoljuk a fához, amelyik a **legolcsóbb éllel** kapcsolódik hozzá.
- A vágás itt mindig: a már fába vett csúcsok | a többi csúcs.

A nyilvántartott adatok

- $k[v]$: a legolcsóbb él súlya, amely v-t a fához kapcsolja.
- $\pi[v]$: ennek az élnek a másik végpontja (a fában lévő csúcs).
- S: a fába még be nem vett csúcsok halmaza (prioritásos sor).

Prim – a pszeudokód

```
függvény Prim(G, start)
  ciklus x ∈ G.Csúcsok
    k[x] ← ∞;   π[x] ← ∅
    S ⇐ x
  ciklus vége
  k[start] ← 0
  ciklus amíg S ≠ ∅
    u ← S.MinKivesz(k)
    ciklus x ∈ u.Szomszédok
      ha x ∈ S és c(u,x) < k[x] akkor
        k[x] ← c(u,x);   π[x] ← u
    ciklus vége
  ciklus vége
  vissza π
függvény vége
```

A pszeudokód olvasata

- S egy **prioritásos sor**. Az `S.MinKivesz(k)` művelet azt az $x \in S$ sorbeli elemet adja vissza, ahol a $k[x]$ érték a legkisebb.
- A visszatérési érték a **szülőket** tartalmazó π adatszerkezet: ebből a feszítőfa élei közvetlenül kiolvashatók.
- Az $x \in S$ feltétel biztosítja, hogy már befűzött csúcsot ne próbáljunk újra csatolni.

Figyeljük meg a különbséget

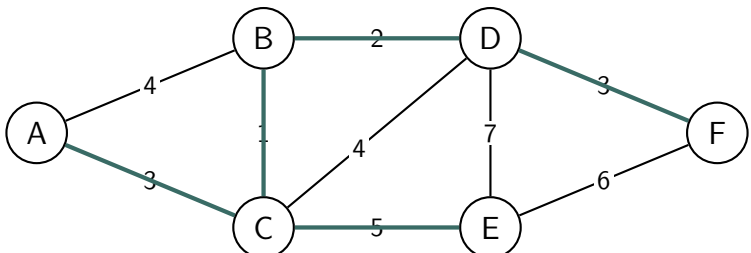
Itt $k[x] \leftarrow c(u, x)$ – **csak az él súlya**.
A legolcsóbb utaknál ott $d[u] + c(u, x)$ állt, azaz a **teljes út** költsége.

Prim – nyomkövetés (A-ból indulva)

kivett	**él**	**frissítések utána**
A	–	$k[B]{=}4, \pi{=}A$; $k[C]{=}3, \pi{=}A$
C	(A, C) 3	$k[B]{=}1, \pi{=}C$; $k[D]{=}4, \pi{=}C$; $k[E]{=}5, \pi{=}C$
B	(C, B) 1	$k[D]{=}2, \pi{=}B$
D	(B, D) 2	$k[F]{=}3, \pi{=}D$
F	(D, F) 3	–
E	(C, E) 5	–

Összsúly: $3 + 1 + 2 + 3 + 5 = \mathbf{14}$ – ugyanaz, mint Kruskalnál.

Prim – az eredmény



Ugyanaz az öt él, mint Kruskalnál – csak más **sorrendben** találtuk meg őket.
Ez a gráf egyetlen minimális feszítőfája.

Prim – futási idő

- A fő ciklus $|V|$-szer fut le, benne egy `MinKivesz` művelettel.
- A belső ciklusok összesen minden élt kétszer néznek meg.

Az S megvalósítása	**Futási idő**
tömb (lineáris minimumkeresés)	$O(\|V\|^2)$
bináris kupac	$O\big((\|V\| + \|E\|) \log \|V\|\big)$
Fibonacci-kupac	$O(\|E\| + \|V\| \log \|V\|)$

Sűrű gráfnál a tömbös változat is versenyképes.

Kruskal vagy Prim?

	Kruskal	**Prim**
mit növel	**erdőt**	egyetlen **fát**
mit választ	a legolcsóbb még jó élt	a fához legolcsóbban csatlakozó csúcsot
segédszerkezet	unió-holvan	prioritásos sor
futási idő	$O(\|E\| \log \|V\|)$	$O(\|E\| + \|V\| \log \|V\|)$
mikor előnyös	**ritka** gráf	**sűrű** gráf

Mindkettő **mohó** algoritmus, és mindkettő a vágási tételre épül.
Az eredmény összsúlya mindig azonos.

Összefoglalás

- A legolcsóbb utak keresése a **fokozatos közelítés** elvére épül: $d[v]$ felső becslés, $\pi[v]$ a megelőző csúcs.
- A **Bellman–Ford-algoritmus** $|V| - 1$ menetben minden élt megvizsgál.
  Negatív élekkel is működik, és a negatív kört felismeri.
- A **minimális feszítőfa** algoritmusai a **vágási tételre** épülnek: egy A-t kikerülő vágás könnyű éle biztonságos.
- **Kruskal** erdőt növeszt élenként, **Prim** egyetlen fát csúcsonként.

Futási idők

Algoritmus	**Futási idő**
Bellman–Ford	$O(\|V\| \cdot \|E\|)$
Kruskal	$O(\|E\| \log \|V\|)$
Prim (bináris kupaccal)	$O\big((\|V\| + \|E\|) \log \|V\|\big)$
Prim (tömbbel)	$O(\|V\|^2)$

Mindhárom algoritmus feltételezi, hogy a gráf a megfelelő típusú:
Bellman–Ford irányított, Kruskal és Prim irányítatlan, összefüggő gráfot vár.

Gyakorló feladatok (1)

1. Futtassa le a Bellman–Ford-algoritmust a példagráfon úgy, hogy
   az éleket **fordított** sorrendben vizsgálja! Hány menet kell?
2. Mutassa meg, hogy alkalmas élsorrend esetén **egyetlen** menet is elég lehet.
3. Adjon példát olyan gráfra, ahol tényleg mind a $|V| - 1$ menetre szükség van!
4. Hogyan módosítaná a Bellman–Ford-algoritmust úgy, hogy a negatív kör **csúcsait** is kiírja?

Gyakorló feladatok (2)

5. Keresse meg a példagráf minimális feszítőfáját úgy, hogy Prim algoritmusát az F csúcsból indítja! Ugyanazt kapja?
6. Mutassa meg: ha minden élsúly különböző, a minimális feszítőfa egyértelmű.
7. Hogyan keresnénk **maximális** összsúlyú feszítőfát?
8. Mi történik Kruskal algoritmusával, ha a gráf **nem összefüggő**?
9. Igaz-e, hogy a gráf legkisebb súlyú éle mindig benne van valamelyik minimális feszítőfában?

Források és ajánlott irodalom

- Szénási Sándor: *Gráfok 2. – Legrövidebb utak, feszítőfák*.
  Óbudai Egyetem, NIK. `nik.uni-obuda.hu/sztf2`
- Házy Attila: *Nemlineáris optimalizálás*, 7. fejezet (Gráfelméleti algoritmusok).
  `gyires.inf.unideb.hu/KMITT/b14/ch07s03.html`
- Cormen, Leiserson, Rivest: *Algoritmusok*. Műszaki Könyvkiadó, 1997.
- R. Bellman: On a routing problem. *Quart. Appl. Math.*, 16, 1958.
- J. B. Kruskal: On the shortest spanning subtree of a graph. *Proc. AMS*, 7(1), 1956.
- R. C. Prim: Shortest connection networks. *Bell Syst. Tech. J.*, 36(6), 1957.